

Full Stack Developer — Step 4

Backend Development

Roadmap objective: Connect a React frontend to a real backend service

Level: Beginner → Intermediate

Last reviewed: September 2026

How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

Learning outcomes

- Client/server
- HTTP
- Node.js
- Express
- Routing
- Middleware
- JSON
- CORS
- Environment config
- Error handling

Core concepts

1. A full-stack request typically flows: browser → HTTP request → backend route → business logic → database → response → UI update.
2. Keep frontend presentation separate from backend secrets and database credentials.
3. CORS is a browser security mechanism controlling cross-origin requests; it is not authentication.
4. API errors should be predictable and useful to the frontend.

Concept map

```
Backend Development
|
+--> Learn the concept
|
+--> Reproduce a small example
|
+--> Apply it in a feature
|
+--> Test failure/edge cases
|
+--> Document the decision
```

Detailed study guide

1. Client/server

Study client/server through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

2. HTTP

Study http through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

3. Node.js

Study node.js through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

4. Express

Study express through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

5. Routing

Study routing through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

6. Middleware

Study middleware through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

Worked example

Frontend → backend flow

```
Browser
|
| GET /api/careers
v
Express route
|
| service/database call
v
MySQL
|
v
JSON response
|
v
React/JS UI update
```

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

Comparison table

Concept	Meaning	Practical use
Frontend	Presentation/user interaction	Browser
Backend	Business logic/API	Server
Database	Persistence	Stores data

Common mistakes

- Putting DB credentials in frontend
- Treating CORS as authentication
- No API error contract
- Tight coupling between UI and DB schema
- No environment separation

Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

Lab 1

- Draw the client/server/database request flow.
- Write down what you expected, what happened, and why.

Lab 2

- Create one Express endpoint.
- Write down what you expected, what happened, and why.

Lab 3

- Call it from the frontend.
- Write down what you expected, what happened, and why.

Lab 4

- Add CORS only when the deployment topology requires it.
- Write down what you expected, what happened, and why.

Lab 5

- Return predictable JSON errors.
- Write down what you expected, what happened, and why.

Lab 6

- Move the API base URL into environment/configuration.
- Write down what you expected, what happened, and why.

Mini-project

Connect a React page to an Express API and MySQL database. Implement one complete feature from form submission to database write and UI update.

Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

What is the client/server boundary?

- Answer structure: definition → example → trade-off/limitation → practical use.

Why should secrets stay on the backend?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is CORS?

- Answer structure: definition → example → trade-off/limitation → practical use.

Where should validation occur?

- Answer structure: definition → example → trade-off/limitation → practical use.

How should API errors be represented?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you configure different environments?

- Answer structure: definition → example → trade-off/limitation → practical use.

Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

Glossary

Term	Your own definition	Example/use
Client	Write this in your own words.	Give one project example.
Server	Write this in your own words.	Give one project example.
API	Write this in your own words.	Give one project example.
Middleware	Write this in your own words.	Give one project example.
CORS	Write this in your own words.	Give one project example.
JSON	Write this in your own words.	Give one project example.
Environment	Write this in your own words.	Give one project example.
Validation	Write this in your own words.	Give one project example.

Term	Your own definition	Example/use
Controller	Write this in your own words.	Give one project example.
Database	Write this in your own words.	Give one project example.

Self-test

- Explain Client/server without using its textbook definition.
- Show a small example of HTTP.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

Final checklist

- I can explain and demonstrate Client/server.
- I can explain and demonstrate HTTP.
- I can explain and demonstrate Node.js.
- I can explain and demonstrate Express.
- I can explain and demonstrate Routing.
- I can explain and demonstrate Middleware.
- I can explain and demonstrate JSON.
- I can explain and demonstrate CORS.
- I can explain and demonstrate Environment config.
- I can explain and demonstrate Error handling.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

Recommended documentation

- <https://nodejs.org/docs/latest/api/>
- <https://expressjs.com/>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP>

Recommended videos

- Sheryians Coding School — Backend Node.js Series: Mastering Node.js Fundamentals
<https://www.youtube.com/watch?v=jhihI4kBAN8> (NEW)
- REST API Tutorial (Existing DB resource 106)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.